

Projeto Final - LionsDev

Turma: LionsDev

Tópicos: levantamento de requisitos, API REST, MongoDB, autenticação, frontend com IA/FlutterFlow, deploy (Render e FlutterFlow), documentação, testes com Postman e apresentação final.

1. Visão geral

Você e seu grupo vão construir uma aplicação full-stack simples, completa e funcional:

- **Backend:** API REST em Node.js + Express + MongoDB + Mongoose, com autenticação e criptografia.
- **Frontend:** telas criadas com apoio de IA ou FlutterFlow, consumindo a API de verdade.
- **Deploy:** backend publicado no Render; frontend publicado no Render ou, se feito no FlutterFlow, pela publicação web do próprio FlutterFlow.
- **Documentação:** levantamento de requisitos, README, coleção de testes e registro do uso de IA.

O foco não é uma interface perfeita. O foco é um sistema que funciona de ponta a ponta: backend organizado, banco integrado, autenticação, frontend chamando as rotas corretas e aplicação publicada.

Um projeto simples e funcional vale mais que um projeto complexo e quebrado. Mas ele ainda precisa parecer um sistema real: um projeto final não deve ser apenas um CRUD isolado.

2. Organização da entrega

- **Grupo:** 2 a 3 integrantes.
- **Tema:** escolhido pelo grupo.
- **Entrega:** repositório no GitHub, levantamento de requisitos, README, coleção do Postman, link do backend no Render, link do frontend publicado, frontend integrado e apresentação.
- **Prazo:** definido em aula.
- **Apresentação:** todos os integrantes devem participar e explicar a própria contribuição.

3. Escopo mínimo obrigatório

Seu projeto está completo quando tiver tudo desta lista.

Backend

- ☐ Pelo menos **4 entidades do domínio**, além de **Usuario**.
- ☐ Pelo menos **2 entidades com CRUD completo**: criar, listar, buscar por ID, atualizar e excluir.
- ☐ Pelo menos **1 relacionamento entre entidades**, usando referência entre documentos no MongoDB.
- ☐ Pelo menos **1 regra de negócio** que envolva mais de uma entidade.
- ☐ Código separado em camadas: **model**, **repository**, **service**, **controller** e **routes**.
- ☐ Middleware de tratamento de erro.
- ☐ Validação dos campos obrigatórios.

Autenticação

- ☐ Cadastro de usuário com senha protegida por hash.
- ☐ Login que valida e-mail/senha e devolve token JWT.
- ☐ Rotas protegidas por middleware de autenticação.

Banco de dados

- ☐ MongoDB Atlas.
- ☐ Dados salvos, buscados, atualizados e removidos de verdade.

Deploy

- ☐ Backend publicado no Render.
- ☐ Frontend publicado (Render ou publicação web do FlutterFlow).
- ☐ Backend publicado usando variáveis de ambiente no painel do Render.
- ☐ Frontend consumindo a URL pública do backend no Render.

Frontend

- ☐ Tela de login consumindo a API.
- ☐ Tela de listagem dos dados vindos do backend.
- ☐ Formulários de cadastro para pelo menos 2 entidades.
- ☐ Ação de editar ou excluir em pelo menos 2 entidades.
- ☐ Tela ou fluxo que mostre dados relacionados entre entidades.
- ☐ Mensagens de sucesso e erro.

O frontend pode ser feito com FlutterFlow, React gerado com IA ou outra abordagem, desde que consuma a API real.

Documentação e organização

- ☐ Levantamento de requisitos.
- ☐ README completo.
- ☐ Registro do uso de IA.
- ☐ Repositório no GitHub.

4. Tecnologias obrigatórias

- Node.js
- Express
- MongoDB
- Mongoose
- dotenv
- bcryptjs
- jsonwebtoken
- Postman
- Render
- GitHub
- Frontend com IA ou FlutterFlow

5. Tema do projeto

Cada grupo escolhe seu próprio tema.

Regras:

- O tema precisa resolver um problema claro.
- Dois grupos não podem ter o mesmo tema.
- O escopo precisa caber no prazo.
- O tema precisa gerar entidades suficientes para parecer um sistema de verdade.

Exemplos de entidades por tema:

Tema	Entidades possíveis
Cafeteria	Produto, Categoria, Cliente, Pedido, ItemPedido
Academia	Aluno, Plano, Matrícula, Treino, Avaliação
Eventos	Evento, Lote, Participante, Inscrição, Check-in
Biblioteca	Livro, Autor, Categoria, Empréstimo, Reserva
Clínica	Paciente, Profissional, Especialidade, Consulta, Prontuário

O **Usuario** do boilerplate continua existindo para autenticação, mas não substitui as entidades do domínio do projeto.

6. Estrutura do Boilerplate

O projeto final deve partir do **Boilerplate Lions Dev** usado em aula:

```
https://github.com/nicolassmotta/boilerplate-lions-dev.git  
git@github.com:nicolassmotta/boilerplate-lions-dev.git
```

A autenticação de usuários já vem pronta no boilerplate. O trabalho do grupo é manter essa base funcionando e adicionar as entidades do domínio seguindo a mesma arquitetura.

```
.  
├── src/  
│   ├── app.js  
│   ├── server.js  
│   ├── config/  
│   │   └── database.js  
│   ├── controllers/  
│   │   ├── auth.controller.js  
│   │   ├── usuario.controller.js  
│   │   ├── [entidade].controller.js  
│   │   └── [outra-entidade].controller.js  
│   ├── middlewares/  
│   │   ├── autenticacao.middleware.js  
│   │   ├── erro.middleware.js  
│   │   └── validarCampos.middleware.js  
│   ├── models/  
│   │   ├── usuario.model.js  
│   │   ├── [entidade].model.js  
│   │   └── [outra-entidade].model.js  
│   ├── repositories/  
│   │   ├── usuario.repository.js  
│   │   ├── [entidade].repository.js  
│   │   └── [outra-entidade].repository.js  
│   ├── routes/  
│   │   ├── auth.routes.js  
│   │   ├── usuario.routes.js  
│   │   ├── [entidade].routes.js  
│   │   └── [outra-entidade].routes.js  
│   ├── services/  
│   │   ├── auth.service.js  
│   │   ├── usuario.service.js  
│   │   ├── [entidade].service.js  
│   │   └── [outra-entidade].service.js  
│   └── utils/  
│       └── criarErro.js  
├── .env.example  
├── .gitignore  
└── LICENSE
```

```
├─ package.json
├─ package-lock.json
├─ render.yaml
└─ README.md
```

O `.env` nunca deve ir para o GitHub com dados reais. O repositório deve ter um `.env.example` com os nomes das variáveis, sem senhas reais.

7. Backend

A API deve seguir o padrão REST. Cada camada tem uma responsabilidade:

- `model` : define o Schema do Mongoose.
- `repository` : conversa diretamente com o MongoDB usando o Model.
- `service` : concentra regras de negócio e validações do fluxo.
- `controller` : recebe `req`, chama o service e responde com `res.status().json()`.
- `routes` : declara os endpoints e aplica middlewares.
- `middleware` : autenticação, validação e tratamento de erros.
- `app.js` : registra middlewares globais, rotas e erro 404.
- `server.js` : carrega `.env`, conecta no MongoDB e inicia o servidor.

Regra principal: controller não acessa o banco diretamente. Quem acessa o banco é o repository. Quem decide regra de negócio é o service.

Rotas mínimas para cada entidade com CRUD completo:

POST	/api/[entidade]	cadastrar
GET	/api/[entidade]	listar
GET	/api/[entidade]/:id	buscar por ID
PATCH	/api/[entidade]/:id	atualizar
DELETE	/api/[entidade]/:id	excluir

Além dos CRUDs, o projeto deve ter pelo menos uma rota de fluxo/regra de negócio conectando entidades.

Exemplos:

POST	/api/pedidos/:id/itens	adicionar item em um pedido
PATCH	/api/consultas/:id/concluir	concluir uma consulta
POST	/api/emprestimos	emprestar um livro para um usuário
PATCH	/api/matriculas/:id/cancelar	cancelar uma matrícula

Rotas de autenticação que já vêm do boilerplate:

POST	/api/auth/cadastro	criar usuário
POST	/api/auth/login	autenticar e retornar token

8. Autenticação

A autenticação deve usar o que já vem pronto no boilerplate.

Rotas já existentes:

```
POST /api/auth/cadastro
POST /api/auth/login
GET /api/usuarios/perfil
PATCH /api/usuarios/perfil
DELETE /api/usuarios/perfil
```

Fluxo esperado no projeto final:

1. Usuário se cadastra -> senha salva com hash.
2. Usuário faz login -> backend valida e retorna token JWT.
3. Frontend guarda o token.
4. Frontend envia Authorization: Bearer TOKEN em rotas protegidas.
5. Middleware valida o token antes de liberar a rota.

As rotas das entidades do projeto devem usar o middleware **autenticar**, importado de:

```
src/middlewares/autenticacao.middleware.js
```

Exemplo:

```
router.post("/", autenticar, EntidadeController.criar);
router.get("/", autenticar, EntidadeController.listar);
router.get("/:id", autenticar, EntidadeController.buscarPorId);
router.patch("/:id", autenticar, EntidadeController.atualizar);
router.delete("/:id", autenticar, EntidadeController.remover);
```

Se o sistema tiver dados individuais de cada usuário, as entidades devem guardar o dono do registro e usar **req.usuario.id** para filtrar o que cada usuário pode ver ou modificar.

9. Banco de dados

Use MongoDB Atlas.

A conexão deve continuar centralizada no arquivo do boilerplate:

```
src/config/database.js
```

Exemplo de `.env` :

```
PORT=3000
MONGO_URI=sua_string_de_conexao_do_mongodb_atlas
JWT_SECRET=sua_chave_secreta
JWT_EXPIRES_IN=1d
BCRYPT_SALT_ROUNDS=10
```

O backend deve salvar, listar, atualizar e excluir dados reais no MongoDB Atlas. Dados fixos em array não serão aceitos.

10. Frontend com IA ou FlutterFlow

Você pode usar IA para criar telas, formulários, estilos e corrigir erros. Também pode usar FlutterFlow para montar visualmente a interface.

O grupo é responsável por entender, testar e ajustar tudo o que for gerado.

O frontend precisa:

- Consumir rotas reais do backend.
- Enviar body com nomes de campos corretos.
- Tratar erro e sucesso.
- Fazer login em `POST /api/auth/login`.
- Enviar `Authorization: Bearer TOKEN` nas rotas protegidas.
- Mostrar dados persistidos no MongoDB.
- Exibir dados de pelo menos 2 entidades do domínio.
- Ter pelo menos 1 tela ou fluxo com relacionamento entre entidades.

Exemplo de requisição autenticada:

```
const API_URL = "https://sua-api.onrender.com"; // em desenvolvimento:
"http://localhost:3000"
```

```
fetch(`${API_URL}/api/[entidade]`, {  
  headers: { Authorization: `Bearer ${token}` },  
});
```

Deixe a URL base em um único lugar do frontend, para trocar entre a API local e a publicada sem alterar cada chamada.

Registro obrigatório do uso de IA:

- Qual IA ou plataforma foi usada.
- Quais partes foram feitas com apoio dela.
- Ajustes feitos pelo grupo.
- Erros encontrados e como foram resolvidos.

11. Deploy

O backend precisa estar publicado no Render. O frontend pode ser publicado no Render ou, se for feito no FlutterFlow, pela publicação web do próprio FlutterFlow.

Backend no Render

O backend deve ser publicado como **Web Service**.

Configurações esperadas:

```
Build Command: npm install  
Start Command: npm start
```

Variáveis de ambiente no painel do Render:

```
MONGO_URI=sua_string_do_mongodb_atlas  
JWT_SECRET=sua_chave_secreta  
JWT_EXPIRES_IN=1d  
BCRYPT_SALT_ROUNDS=10  
NODE_ENV=production
```

O backend deve funcionar usando `process.env.PORT`, como no boilerplate. Não coloque `.env` no GitHub.

Frontend

O frontend deve estar publicado e consumir a URL pública do backend.

Se o frontend for React/Vite, publique como **Static Site**:

```
Build Command: npm install && npm run build  
Publish Directory: dist
```

Se o frontend for feito no FlutterFlow, publique pela opção de publicação web do próprio FlutterFlow. Nesse caso, a API Call deve usar a URL do backend no Render.

O frontend não deve chamar **localhost** na versão publicada. Ele precisa usar a URL do backend no Render. Se o navegador bloquear as requisições com erro de CORS, confirme que o backend libera requisições vindas do domínio do frontend.

Links obrigatórios na entrega

```
Link do repositório:  
Link do backend no Render:  
Link do frontend publicado:
```

12. Levantamento de requisitos

Antes de programar, entregue o levantamento de requisitos usando o template deste módulo.

O documento precisa responder:

```
Problema: qual problema o sistema resolve.  
Público-alvo: quem usa o sistema.  
Solução: o que o sistema faz.  
Escopo: o que entra e o que não entra.  
Entidades: quais dados serão salvos.  
Relacionamentos: como as entidades se conectam.  
Telas/endpoints: como o usuário interage com o sistema.  
Requisitos funcionais: o que o sistema deve fazer.  
Requisitos não funcionais: como o sistema deve se comportar.  
Regras de negócio: condições específicas do domínio.
```

Os requisitos devem ser coerentes com o tema do grupo, não cópia dos exemplos. O que estiver no levantamento precisa aparecer no sistema ou ser explicado como fora do escopo.

13. README obrigatório

O README do projeto final deve conter:

```
# Nome do Projeto

## Descrição
Explicação curta do sistema.

## Tecnologias
Node.js, Express, MongoDB, Mongoose, frontend usado, IA usada.

## Funcionalidades
Lista das principais funcionalidades.

## Como rodar
Passo a passo: npm install, criar .env, npm start.

## Variáveis de ambiente
Explicação de cada variável do .env.

## Rotas da API
Lista das principais rotas.

## Frontend
Como acessar, testar e integrar com a API.

## Deploy
Link do backend no Render e link do frontend publicado.

## Postman
Como importar/testar a coleção de requisições.

## Uso de IA
Como a IA foi usada e quais ajustes foram feitos.

## Integrantes
Nomes do grupo.
```

14. Divisão de trabalho

O trabalho é em grupo, mas a nota tem componente individual.

- Cada integrante deve ser dono de pelo menos uma funcionalidade.
- A contribuição será avaliada pelo histórico de commits e pela apresentação.
- Integrante que não contribuir ou não souber explicar sua parte pode receber nota individual menor.

Exemplo de divisão:

Integrante 1: backend de parte das entidades
Integrante 2: frontend e integração com a API
Integrante 3: levantamento de requisitos, README e Postman
Todos: testes no Postman e apresentação

Cada integrante precisa conseguir explicar:

O que fez.
Quais arquivos alterou.
Qual rota ou tela ficou sob sua responsabilidade.
Quais erros encontrou e como resolveu.

| 15. Testes das rotas

Todas as rotas devem ser testadas antes da entrega com Postman. O frontend também deve ser testado, mas ele não substitui a validação das rotas no Postman.

Teste as rotas usando a URL local e também a URL publicada no Render.

Testem:

- Cadastro de usuário.
- Login.
- Listagem das entidades com CRUD.
- Busca por ID.
- Cadastro das entidades com CRUD.
- Atualização das entidades com CRUD.
- Exclusão das entidades com CRUD.
- Rota de relacionamento ou regra de negócio entre entidades.
- Rota protegida sem token.
- Rota protegida com token inválido.
- Campos obrigatórios vazios.
- ID inexistente.

Exportem a coleção do Postman e incluam no repositório do projeto.

| 16. Apresentação final

Todos os integrantes participam.

A apresentação deve cobrir:

- Problema e público-alvo.
- Requisitos principais.
- Entidades, relacionamentos e organização do backend.
- Demonstração prática do CRUD e login.
- Demonstração de uma regra de negócio entre entidades.
- Dados persistindo no MongoDB.
- Frontend consumindo a API.
- Backend publicado no Render e frontend publicado.
- Como a IA/FlutterFlow ajudou.
- Dificuldades e próximos passos.

17. Checklist final

- ☐ Backend roda sem erro.
- ☐ Frontend roda sem erro.
- ☐ Backend está publicado no Render.
- ☐ Frontend está publicado (Render ou FlutterFlow).
- ☐ Frontend publicado consome a URL pública do backend.
- ☐ MongoDB conectado e salvando de verdade.
- ☐ Existem pelo menos 4 entidades do domínio além de Usuario.
- ☐ Pelo menos 2 entidades possuem CRUD completo.
- ☐ Existe pelo menos 1 relacionamento entre entidades.
- ☐ Existe pelo menos 1 regra de negócio envolvendo mais de uma entidade.
- ☐ Cadastro e login funcionam.
- ☐ Senha é salva com hash.
- ☐ Token JWT é gerado no login.
- ☐ Rotas das entidades usam autenticação.
- ☐ Frontend consome a API.
- ☐ Rotas foram testadas no Postman.
- ☐ Coleção do Postman foi incluída no repositório.
- ☐ README está completo.
- ☐ README possui links do backend e do frontend publicados.
- ☐ .env.example está no repositório sem dados sensíveis.
- ☐ .env não foi enviado ao GitHub.
- ☐ Levantamento de requisitos foi entregue.
- ☐ Uso de IA foi documentado.
- ☐ Todos os integrantes sabem explicar o projeto.